

Sprog og oversættelse

Lexical analysis: From NFA to DFA; Minimizing a DFA

Part 5: A horror story

Hans Hüttel

(using content by Cosmin Oancea)

Creative Commons License
Attribution Non Commercial 4.0 International
(CC-BY-NC-4.0)



The ad hoc monster

The ad hoc monster often rears its ugly head when constructions on automata appear. Some people say

- *The subset construction is too complicated for me to wrap my head around!*
- *However, **I am a very competent person** so I can always find a DFA, given an NFA using some form of ad hoc'ery that I will invent on the day that I need to do this.*

Fighting the ad hoc monster

- The principles behind the subset construction are simple – in fact there is only one: We keep track of which states the NFA can reach, whenever we read a symbol. A competent person can learn to use this algorithm – all it takes is some practice.
- We can implement the subset construction but not ad hoc approaches. Lexer generators use the subset construction, not ad hoc'ery.
- And as always, ad hoc approaches do not work.

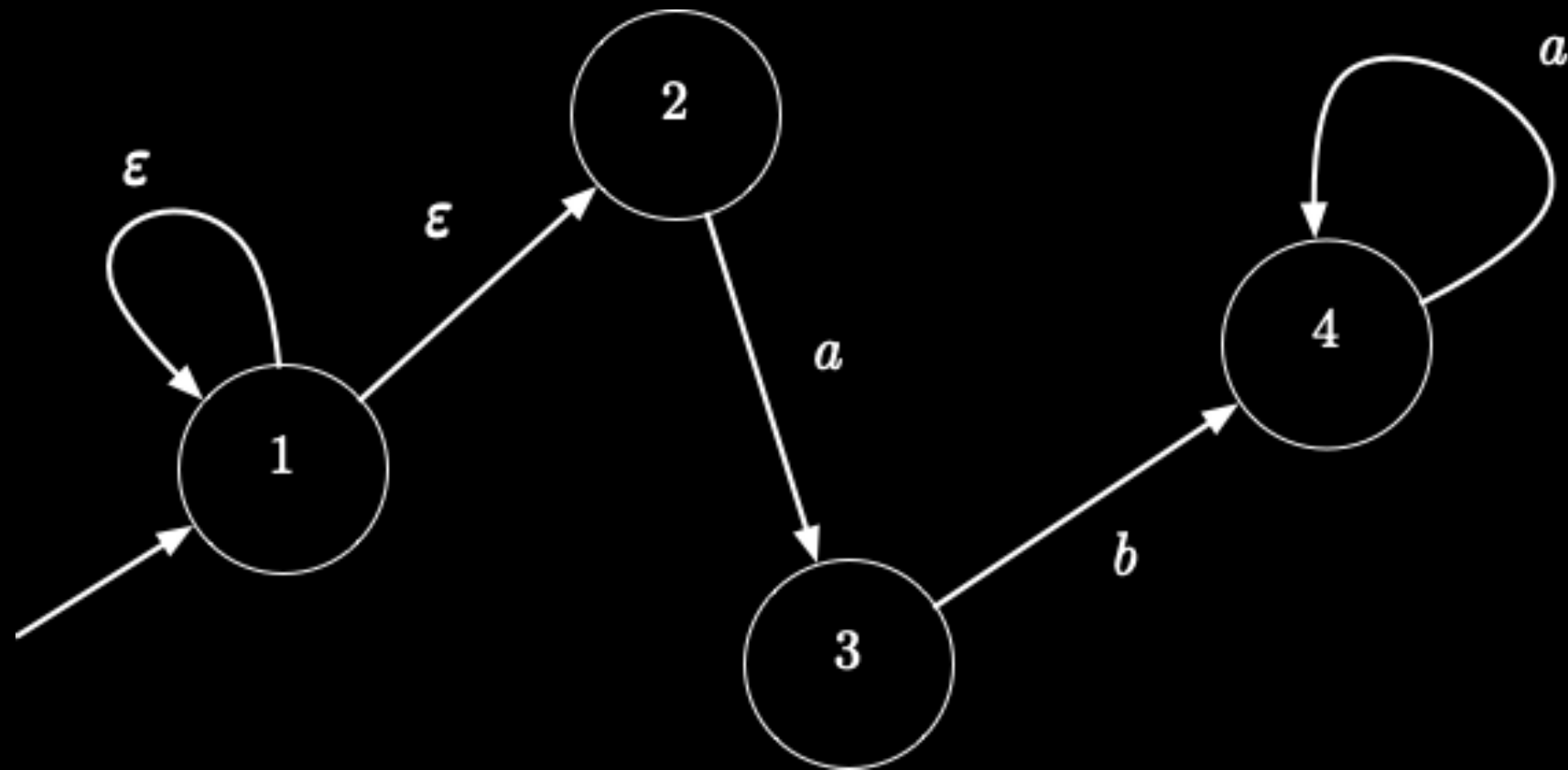
The ϵ -closure and the start state

Many who are otherwise familiar with the subset construction nevertheless forget one thing:

We must be careful about getting the start state right! If there are ϵ -transitions from the start state, we can start reading the input in any state that can be reached in this way.

The ϵ -closure and the start state

In this NFA, we have that $\hat{e}(\{1\}) = \{1, 2\}$.



So when we build a DFA using the subset construction, the new start state is $\{1, 2\}$, **not** $\{1\}$.